

共通

適宜コミットとmainブランチへのマージPRを行うこと。

相談・質問はいつでも受け付けます。画面共有できたほうがわかりやすいと思うので通話も歓迎。

保存・復元機能(+検証)

- `/src/validation/validators.ts`: 保存条件を満たすかどうかを検証する関数群
- `/src/services/local-storage.service.ts`: 保存・復元の処理を行うサービス
 - `LocalStorageService.save(state: AppState): boolean`: 保存条件を満たすことを `validators.ts` の関数を使って検証し、満たす場合は `localStorage` に保存し `true` を返す。保存条件を満たさない場合は保存せず `false` を返す。
 - `LocalStorageService.load(): RestoreResult`: `localStorage` から復元する処理を行う。

一言コメント

`save()` 内で検証もやる、あと修正できるものは `save()` 内で修正を行って、修正できない場合に `false` を返すようにする。多分そこはまだ実装してないはず

Undo・Redo機能

- `/src/history/history-manager.ts`: Undo・Redoの履歴管理を行う
 - `class HistoryEntry`: 履歴の1エントリを表すクラス
 - `class HistoryManager`: Undo・Redoの履歴管理を行うクラス
 - `HistoryManager.record(afterState: AppState)`: 現在の状態を履歴に記録する
 - `HistoryManager.undo(): AppState | null`: Undoを行い、1つ前の状態を返す。Undoできない場合は `null` を返す。
 - `HistoryManager.redo(): AppState | null`: Redoを行い、1つ後の状態を返す。Redoできない場合は `null` を返す。

一言コメント

- `HistoryEntry` クラスは内容をもう一度詰め直す必要あり。
- 大体の実装イメージはこんな感じ
 - `record()`: 現在の状態をパースして、`undoStack` の最後の状態と比較して差分を取得、その差分を `HistoryEntry` に変換して `undoStack` に追加する。
 - `undo()`: `undoStack` の最後の `HistoryEntry` を取り出して、そこに記録されている差分を現在の状態に適用する。適用後の `AppState` (=差分だけではなく全体の状態) を返す。
 - `redo()`: `redoStack` の最後の `HistoryEntry` を取り出して、そこに記録されている差分を現在の状態に適用する。適用後の `AppState` (=差分だけではなく全体の状態) を返す。
 - また、`undo/redo` 後の `Entry` の移動先・破棄の有無に注意すること。

検索・フィルター機能

- コミット5db46adの通り、`FilterService.apply()`を公開APIとし、それぞれ`FilterSource`、`FilterCriteria`、`FilterResult`を引数・戻り値として受け取るようにする。
- `FilterService.apply()`の中ではディープコピーは行う必要はないが、
 - タスクを変更しない
 - 入力順を維持する
 - 入力配列自体を変更しない
 - 戻り値の配列は新しく作る
 - IDからの検索は`/src/domain/entity-finders.ts`の`findCanvasById()`、`findCanvasByTaskId()`、`findTaskById()`、`findConnectionById()`を利用する